

```
PS D:\myk\flu-no-crud\flutter_app> flutter --version
Flutter 3.24.2 • channel stable • https://github.com/flutter/flutter.git
Framework • revision 4cf269e36d (2 years, 1 month ago) • 2024-09-03 14:30:00 -0700
Engine • revision a6bd3f1de1
Tools • Dart 3.5.2 • DevTools 2.37.2
```

```
PS D:\myk\flu-no-crud\backend> node -v
v22.15.0
```

```
flutter_node_crud/
├── backend/
│   ├── server.js
│   ├── package.json
│   └── .env
└── flutter_app/
    └── lib/
        ├── main.dart
        ├── models/
        │   └── user.dart
        ├── services/
        │   └── api_service.dart
        └── screens/
            ├── home_screen.dart
            └── user_form_screen.dart
```

Backend — Node.js + Express + MongoDB

Create the backend:

```
mkdir backend
cd backend
npm init -y
npm install express mongoose cors dotenv
npm install --save-dev nodemon
```

Update package.json:

```
{
  "name": "flutter-crud-api",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^16.0.0",
    "express": "^5.0.0",
    "mongoose": "^8.0.0"
  },
  "devDependencies": {
    "nodemon": "^3.0.0"
  }
}
```

.env

Create:

```
backend/.env
```

Add:

```
PORT=3000
```

```
MONGO_URI=mongodb://127.0.0.1:27017/flutter_crud
```

If you're using MongoDB Atlas, replace `MONGO_URI` with your Atlas connection string.

2. server.js

Create:

```
backend/server.js
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
require("dotenv").config();

const app = express();

// Middleware
app.use(cors());
app.use(express.json());

// MongoDB connection
mongoose
  .connect(process.env.MONGO_URI)
  .then(() => {
    console.log("MongoDB connected");
  })
  .catch((error) => {
    console.error("MongoDB connection error:", error);
  });

// User model
const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
      trim: true,
    },
    email: {
      type: String,
      required: true,
      trim: true,
    },
  },
  {
    timestamps: true,
  }
);

const User = mongoose.model("User", userSchema);

// Test route
app.get("/", (req, res) => {
  res.json({
    message: "Flutter CRUD API is running",
  });
});

// CREATE
app.post("/api/users", async (req, res) => {
  try {
    const { name, email } = req.body;

    if (!name || !email) {
      return res.status(400).json({
```

```

        message: "Name and email are required",
    });
}

const user = await User.create({
    name,
    email,
});

res.status(201).json(user);
} catch (error) {
    res.status(500).json({
        message: "Failed to create user",
        error: error.message,
    });
}
});

// READ ALL
app.get("/api/users", async (req, res) => {
    try {
        const users = await User.find().sort({ createdAt: -1 });

        res.json(users);
    } catch (error) {
        res.status(500).json({
            message: "Failed to fetch users",
            error: error.message,
        });
    }
});

// READ ONE
app.get("/api/users/:id", async (req, res) => {
    try {
        const user = await User.findById(req.params.id);

        if (!user) {
            return res.status(404).json({
                message: "User not found",
            });
        }

        res.json(user);
    } catch (error) {
        res.status(500).json({
            message: "Failed to fetch user",
            error: error.message,
        });
    }
});

// UPDATE
app.put("/api/users/:id", async (req, res) => {
    try {
        const { name, email } = req.body;

        if (!name || !email) {
            return res.status(400).json({
                message: "Name and email are required",
            });
        }

        const user = await User.findByIdAndUpdate(
            req.params.id,
            {
                name,
                email,
            },

```

```

    {
      new: true,
      runValidators: true,
    }
  );

  if (!user) {
    return res.status(404).json({
      message: "User not found",
    });
  }

  res.json(user);
} catch (error) {
  res.status(500).json({
    message: "Failed to update user",
    error: error.message,
  });
}
});

// DELETE
app.delete("/api/users/:id", async (req, res) => {
  try {
    const user = await User.findByIdAndDelete(req.params.id);

    if (!user) {
      return res.status(404).json({
        message: "User not found",
      });
    }

    res.json({
      message: "User deleted successfully",
    });
  } catch (error) {
    res.status(500).json({
      message: "Failed to delete user",
      error: error.message,
    });
  }
});

// Start server
const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});

```

Start it:

```
npm run dev
```

You should see:

```

MongoDB connected
Server running on http://localhost:3000

```

Test in browser:

```
http://localhost:3000
```

You should get:

```

{
  "message": "Flutter CRUD API is running"
}

```

Flutter

```
D:\myk\flu-no-crud
flutter create flutter_app
```

1. Install HTTP package

From:
cd D:\myk\flu-no-crud\flutter_app
run:
flutter pub add http

2. Create folders

Inside lib, create:

```
lib/
├── main.dart
├── models/
│   └── user.dart
├── services/
│   └── api_service.dart
└── screens/
    ├── home_screen.dart
    └── user_form_screen.dart
```

You can create them manually in VS Code.

3. lib/models/user.dart

```
class User {
  final String id;
  final String name;
  final String email;

  User({
    required this.id,
    required this.name,
    required this.email,
  });

  factory User.fromJson(Map<String, dynamic> json) {
    return User(
      id: json['_id'] ?? '',
      name: json['name'] ?? '',
      email: json['email'] ?? '',
    );
  }

  Map<String, dynamic> toJson() {
    return {
      'name': name,
      'email': email,
    };
  }
}
```

4. lib/services/api_service.dart

```
import 'dart:convert';

import 'package:http/http.dart' as http;

import '../models/user.dart';

class ApiService {
  // Android Emulator

  static const String baseUrl = 'http://localhost:3000/api';
```

```

// If using iOS Simulator:
// static const String baseUrl = 'http://localhost:3000/api';

// If using a physical Android phone:
// static const String baseUrl = 'http://192.168.1.10:3000/api';

// GET ALL USERS
static Future<List<User>> getUsers() async {
  final response = await http.get(
    Uri.parse('$baseUrl/users'),
  );

  if (response.statusCode == 200) {
    final List data = jsonDecode(response.body);

    return data
      .map(
        (json) => User.fromJson(json),
      )
      .toList();
  }

  throw Exception(
    'Failed to load users',
  );
}

// CREATE USER
static Future<User> createUser(
  String name,
  String email,
) async {
  final response = await http.post(
    Uri.parse('$baseUrl/users'),
    headers: {
      'Content-Type': 'application/json',
    },
    body: jsonEncode({
      'name': name,
      'email': email,
    }),
  );

  if (response.statusCode == 201) {
    return User.fromJson(
      jsonDecode(response.body),
    );
  }

  throw Exception(
    'Failed to create user',
  );
}

// UPDATE USER
static Future<User> updateUser(
  String id,
  String name,
  String email,
) async {
  final response = await http.put(
    Uri.parse('$baseUrl/users/$id'),
    headers: {
      'Content-Type': 'application/json',
    },
    body: jsonEncode({
      'name': name,
      'email': email,
    }),
  );
}

```

```

);

if (response.statusCode == 200) {
  return User.fromJson(
    jsonDecode(response.body),
  );
}

throw Exception(
  'Failed to update user',
);
}

// DELETE USER
static Future<void> deleteUser(
  String id,
) async {
  final response = await http.delete(
    Uri.parse('$baseUrl/users/$id'),
  );

  if (response.statusCode != 200) {
    throw Exception(
      'Failed to delete user',
    );
  }
}
}

```

5. lib/screens/home_screen.dart

```

import 'package:flutter/material.dart';

import '../models/user.dart';
import '../services/api_service.dart';
import 'user_form_screen.dart';

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  List<User> users = [];

  bool isLoading = true;

  @override
  void initState() {
    super.initState();

    loadUsers();
  }

  // GET USERS
  Future<void> loadUsers() async {
    setState(() {
      isLoading = true;
    });

    try {
      final result = await ApiService.getUsers();

      if (!mounted) return;

      setState(() {
        users = result;
      });
    } catch (e) {
      // Handle error
    }
  }
}

```

```

    });
  } catch (error) {
    if (!mounted) return;

    showMessage(
      'Failed to load users',
    );
  } finally {
    if (!mounted) return;

    setState(() {
      isLoading = false;
    });
  }
}

// DELETE USER
Future<void> deleteUser(User user) async {
  try {
    await ApiService.deleteUser(
      user.id,
    );

    showMessage(
      'User deleted successfully',
    );

    await loadUsers();
  } catch (error) {
    showMessage(
      'Failed to delete user',
    );
  }
}

// DELETE CONFIRMATION
Future<void> confirmDelete(User user) async {
  final confirmed = await showDialog<bool>(
    context: context,
    builder: (context) {
      return AlertDialog(
        title: const Text(
          'Delete User',
        ),
        content: Text(
          'Are you sure you want to delete ${user.name}?',
        ),
        actions: [
          TextButton(
            onPressed: () {
              Navigator.pop(
                context,
                false,
              );
            },
            child: const Text(
              'Cancel',
            ),
          ),
          ElevatedButton(
            onPressed: () {
              Navigator.pop(
                context,
                true,
              );
            },
            child: const Text(
              'Delete',
            ),
          ),
        ],
      );
    },
  );
}

```

```

        ),
      ],
    );
  },
);

if (confirmed == true) {
  await deleteUser(user);
}
}

// OPEN ADD SCREEN
Future<void> openAddScreen() async {
  final result = await Navigator.push(
    context,
    MaterialPageRoute(
      builder: (context) {
        return const UserFormScreen();
      },
    ),
  );
};

if (result == true) {
  await loadUsers();
}
}

// OPEN EDIT SCREEN
Future<void> openEditScreen(User user) async {
  final result = await Navigator.push(
    context,
    MaterialPageRoute(
      builder: (context) {
        return UserFormScreen(
          user: user,
        );
      },
    ),
  );
};

if (result == true) {
  await loadUsers();
}
}

void showMessage(String message) {
  if (!mounted) return;

  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(
      content: Text(message),
    ),
  );
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text(
        'Flutter CRUD',
      ),
      centerTitle: true,
    ),

    body: RefreshIndicator(
      onRefresh: loadUsers,
      child: isLoading
        ? const Center(

```

```
        child: CircularProgressIndicator(),
      ),
    : users.isEmpty
      ? ListView(
          children: const [
            SizedBox(
              height: 250,
            ),
            Center(
              child: Text(
                'No users found',
                style: TextStyle(
                  fontSize: 18,
                ),
              ),
            ),
          ],
        )
      : ListView.builder(
          padding: const EdgeInsets.all(
            12,
          ),
          itemCount: users.length,
          itemBuilder: (
            context,
            index,
          ) {
            final user = users[index];

            return Card(
              margin: const EdgeInsets.only(
                bottom: 10,
              ),
              child: ListTile(
                leading: CircleAvatar(
                  child: Text(
                    user.name.isNotEmpty
                      ? user.name[0]
                        .toUpperCase()
                      : '?',
                  ),
                ),

                title: Text(
                  user.name,
                  style: const TextStyle(
                    fontWeight:
                      FontWeight.bold,
                  ),
                ),

                subtitle: Text(
                  user.email,
                ),

                trailing: Row(
                  mainAxisAlignment:
                    MainAxisAlignment.min,
                  children: [
                    IconButton(
                      icon: const Icon(
                        Icons.edit,
                        color: Colors.blue,
                      ),
                      onPressed: () {
                        openEditScreen(
                          user,
                        );
                      },
                    ),

```



```

@override
void initState() {
  super.initState();

  if (widget.user != null) {
    nameController.text =
      widget.user!.name;

    emailController.text =
      widget.user!.email;
  }
}

@override
void dispose() {
  nameController.dispose();
  emailController.dispose();

  super.dispose();
}

Future<void> saveUser() async {
  if (!formKey.currentState!.validate()) {
    return;
  }

  setState(() {
    isLoading = true;
  });

  try {
    if (isEdit) {
      // UPDATE
      await ApiService.updateUser(
        widget.user!.id,
        nameController.text.trim(),
        emailController.text.trim(),
      );
    } else {
      // CREATE
      await ApiService.createUser(
        nameController.text.trim(),
        emailController.text.trim(),
      );
    }

    if (!mounted) return;

    Navigator.pop(
      context,
      true,
    );
  } catch (error) {
    if (!mounted) return;

    ScaffoldMessenger.of(context)
      .showSnackBar(
        SnackBar(
          content: Text(
            isEdit
              ? 'Failed to update user'
              : 'Failed to create user',
          ),
        ),
      );
  } finally {
    if (!mounted) return;

    setState(() {

```

```

        isLoading = false;
    });
}
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text(
        isEdit
          ? 'Edit User'
          : 'Add User',
      ),
    ),
    body: SingleChildScrollView(
      padding: const EdgeInsets.all(
        20,
      ),
      child: Form(
        key: formKey,

        child: Column(
          children: [
            // NAME
            TextFormField(
              controller: nameController,

              decoration:
                const InputDecoration(
                  labelText: 'Name',
                  border:
                    OutlineInputBorder(),
                  prefixIcon:
                    Icon(Icons.person),
                ),

              validator: (value) {
                if (value == null ||
                    value.trim().isEmpty) {
                  return 'Please enter name';
                }

                return null;
              },
            ),

            const SizedBox(
              height: 20,
            ),

            // EMAIL
            TextFormField(
              controller:
                emailController,

              keyboardType:
                TextInputType.emailAddress,

              decoration:
                const InputDecoration(
                  labelText: 'Email',
                  border:
                    OutlineInputBorder(),
                  prefixIcon:
                    Icon(Icons.email),
                ),

```

```

        validator: (value) {
          if (value == null ||
              value.trim().isEmpty) {
            return 'Please enter email';
          }

          if (!value.contains('@')) {
            return 'Please enter a valid email';
          }

          return null;
        },
      ),

      const SizedBox(
        height: 30,
      ),

      // SAVE BUTTON
      SizedBox(
        width: double.infinity,
        height: 50,

        child: ElevatedButton(
          onPressed:
            isLoading
              ? null
              : saveUser,

          child: isLoading
            ? const SizedBox(
                width: 24,
                height: 24,
                child:
                  CircularProgressIndicator(
                    strokeWidth: 2,
                  ),
              )
            : Text(
                isEdit
                  ? 'Update User'
                  : 'Add User',
              ),
        ),
      ),
    ],
  ),
),
),
),
),
);
}
}

```

7. Replace `lib/main.dart`

Delete the existing contents and put:

```

import 'package:flutter/material.dart';

import 'screens/home_screen.dart';

void main() {
  runApp(
    const MyApp(),
  );
}

class MyApp extends StatelessWidget {
  const MyApp({

```

```
        super.key,  
    });  
  
    @override  
    Widget build(BuildContext context) {  
        return MaterialApp(  
            debugShowCheckedModeBanner: false,  
  
            title: 'Flutter CRUD',  
  
            theme: ThemeData(  
                colorScheme:  
                    ColorScheme.fromSeed(  
                        seedColor: Colors.blue,  
                    ),  
                useMaterial3: true,  
            ),  
  
            home: const HomeScreen(),  
        );  
    }  
}
```

8. Run the app

First make sure your Node backend is running.

Terminal 1

```
cd D:\myk\flu-no-crud\backend  
npm run dev
```

You should see:

```
MongoDB connected  
Server running on http://localhost:3000
```

Terminal 2

```
cd D:\myk\flu-no-crud\flutter_app  
flutter run
```